

SECURE YOUR **JAVASCRIPT** APPLICATION



Cross-Site Scripting (XSS)

What is XSS?

XSS is a security vulnerability where an attacker injects malicious scripts into content that users trust, often leading to unauthorized access to sensitive data or hijacking user sessions.

Types of XSS:

- **Stored XSS:** The malicious script is stored on the server and executed when the victim loads the page.
- **Reflected XSS:** The malicious script is reflected off a web server (e.g., in a URL query string) and executed immediately.
- **DOM-based XSS:** The vulnerability occurs when JavaScript modifies the DOM and executes malicious content.



How to Prevent XSS:

- **Sanitize Inputs:** Always sanitize user inputs using libraries like DOMPurify.

```
JS index.js

// Example: Sanitizing user input
const sanitizedInput = DOMPurify.sanitize(userInput);
```

- **Escape Outputs:** Use functions to escape special characters when displaying user data in the HTML.

```
JS index.js

// Example: Escaping user inputs
const safeText = userInput.replace(/</g, '&lt;').replace(/>/g, '&gt;');
```

- **Use Content-Security-Policy (CSP):** Implement CSP to restrict what resources can be loaded and executed.

```
index.html

<meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self'">
```



Cross-Site Request Forgery (CSRF)

What is CSRF?

CSRF is an attack that tricks a user into executing unwanted actions on a different site where they are authenticated.

How CSRF Works:

1. The attacker sends a request from their site, pretending to be the user.
2. The victim is tricked into sending the request while logged into a target site.
3. The server executes the action on behalf of the attacker without realizing the user's intention.



How to Prevent CSRF:

- **Use Anti-CSRF Tokens:** These are tokens that are included in each request to verify that the request came from the legitimate site.

```
Js index.js

// Example: Sending CSRF token with requests
fetch('/api/endpoint', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'CSRF-Token': csrfToken, // Add CSRF token to headers
  },
  body: JSON.stringify(data),
});
```

- **SameSite Cookies:** Set the SameSite attribute on cookies to prevent sending cookies on cross-site requests.

```
Js index.js

// Example: Setting SameSite cookie
document.cookie = "sessionId=abc123; SameSite=Strict";
```

- **Double-Submit Cookies:** Use a combination of cookies and request parameters for verification.



Cross-Origin Resource Sharing (CORS)

What is CORS ?

CORS is a security feature that allows servers to control how resources on the web are requested from other domains.

CORS Policy:

- **Same-Origin Policy (SOP):** By default, a web page can only request resources from the same domain, protocol, and port.
- CORS allows web applications to request resources from domains other than the origin.



How to Set Up CORS:

- **Allow CORS Requests:** Configure your server to allow cross-origin requests.

```
JS index.js

// Example: Enabling CORS on an Express.js server
const cors = require('cors');
app.use(cors({
  origin: 'https://yourfrontenddomain.com', // specify allowed domain
  methods: ['GET', 'POST'],
  allowedHeaders: ['Content-Type', 'Authorization'],
}));
```

- **Handle Preflight Requests:** Some requests require an OPTIONS preflight to check if the actual request is safe.

```
JS index.js

app.options('/api/endpoint', cors());
```



Password Management and Authentication

Password Storage:

- Never store plain text passwords. Always hash passwords using secure algorithms like bcrypt or argon2.

```
index.js

// Example: Hashing a password with bcrypt
const bcrypt = require('bcrypt');
const hashedPassword = await bcrypt.hash(password, 10);
```

Secure Authentication:

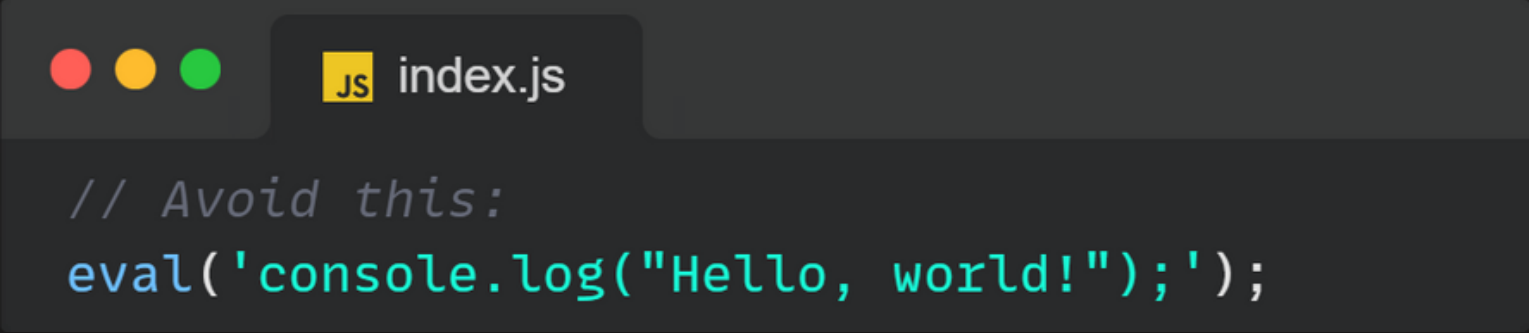
- Use OAuth2 or JWT (JSON Web Tokens) for secure authentication mechanisms.
 - JWT allows safe transmission of claims (e.g., user identity) in a token that can be verified on the server.



Secure JavaScript Practices

General Best Practices:

- **Avoid `eval()` and `setTimeout()` with string arguments:** These functions can execute arbitrary JavaScript, making the application vulnerable to injection attacks.



```
// Avoid this:  
eval('console.log("Hello, world!");');
```

Always Use HTTPS :

- SSL/TLS Encryption: Ensure your site uses HTTPS to protect data from being intercepted by attackers.
 - Always redirect HTTP to HTTPS using server-side redirection.



```
// Example: HTTP to HTTPS redirection  
app.use((req, res, next) => {  
  if (req.protocol !== 'https') {  
    return res.redirect('https://' + req.headers.host + req.url);  
  }  
  next();  
});
```





Hopefully You Found It Usefull!

“Be sure to save this post so you
can come back to it later

like

Comment

Share